

System Overview

ThreatDetect is an insider-threat detection prototype that was built on a Streamlit web application. It utilises an XGBoost classifier and an Isolation Forest anomaly detector to analyse employee related features and find any malicious behaviour at both individual and organizational level.

Key Abilities

- Batch threat detection from uploaded CSV files
- Organisational risk summary with metrics and charts
- Per-record explainability through the use of SHAP values
- Exploratory Data Analysis for uploaded or bundled datasets
- CSV export of prediction results

Core components:

- Streamlit_app.py - main streamlit application
- AI_Model_Code/insider threat_model.pkl - a model package with XGBoost, scaler, encoders and Isolation Forest
- AI_Model_Code/insider threat_clean_datset.csv - dataset used for the training and placeholder for EDA
- assets/app_styling - Streamlit Styling overrides

Software Requirements and Dependencies

The project dependencies are listed in requirements.txt.

The Python packages are required:

- streamlit
- numpy
- pandas
- scikit-learn
- matplotlib
- seaborn

- plotly
- xgboost
- shap
- pickle
- Os

Additional environment notes:

- Python 3.8 and above is extremely recommended.
- The model file is serialised and is compatible with scikit-learn 1.6.1 serialisation, Newer library versions should be used carefully

Installation Instructions

1. Clone the repository:

```
git clone https://github.com/jazbengu/ThreatDetect.git
cd ThreatDetect
```

2. Create and activate a virtual environment (recommended):

```
python3 -m venv venv
source venv/bin/activate
```

3. Install dependencies:

```
pip install -r requirements.txt
```

4. Confirm the repository contains the following key files:

```
- streamlit_app.py
- requirements.txt
- AI_Model_Code/insider_threat_model.pkl
- AI_Model_Code/insider_threat_clean_dataset.csv
- assets/app_styling.css
```

Alternatively, the prototype can be accessed via: <https://threatdetectcos720.streamlit.app/>

Deployment and Execution Instructions

To launch the prototype locally

```
Streamlit run streamlit_app.py
```

When Streamlit starts open the local URL displayed in the terminal

App Workflow:

- The sidebar is there to choose one of two pages
 - Organisation Search via CSV
 - Exploratory Data Analysis
- Upload a CSV file in Organisational Search Via CSV to run batch threat detection
- Use Exploratory Data Analysis page to preview the dataset, numerical and categorical visualisation and a quick anomaly scan

Source Code Documentation

`streamlit_app.py`

This is the main application entry point for ThreatDetect.

Key functions:

- `get_styling(style_path)`: loads CSS styling from `assets/app_styling.css`.
- `load_model()` — loads `AI_Model_Code/insider_threat_model.pkl` and ensures the SHAP explainer is present.
- `validate_input_columns(df, required_columns)` — checks the uploaded dataset for required raw columns.

- `prepare_features(df, model_package)` — cleans raw data, engineers derived ratios, label-encodes categories, scales numeric features, and appends the Isolation Forest score.
- `results_explainability(model_package, x_append_row, iso_score, original_row, feature_names, threshold)` — computes SHAP values and returns human-readable explanations for a selected employee.
- `main()` — controls the Streamlit page structure, user upload flow, model inference, visualization, download, and EDA pages.

Page behavior:

- Organisational Search via CSV uploads a dataset, performs batch predictions with XGBoost, displays counts and charts, and provides per-record SHAP explanation.
- Exploratory Data Analysis loads either a user CSV or bundled sample data, presents summary statistics, displays numeric/categorical plots, and offers a quick Isolation Forest anomaly scan.

`AI_Model_Code/insider_threat_model.pkl`

This file contains a serialized dictionary with the trained model pipeline and metadata.

Saved package contents include:

`xgb_model`: trained `XGBClassifier`

`iso_forest`: trained `IsolationForest`

`scaler`: `StandardScaler`

`label_encoders`: dictionary of `LabelEncoder` objects for categorical columns

`feature_columns`: ordered list of model input features

`cat_cols`: categorical column names

`bin_cols`: binary column names

`num_cols`: numeric column names

`best_threshold`: detection threshold used for classifying risk

feat_importance: feature importance data frame

AI_Model_Code/insider_threat_clean_dataset.csv

A sample dataset used by the EDA page if the user does not upload their own CSV.

assets/app_styling.css

Custom CSS for Streamlit page styling and layout.

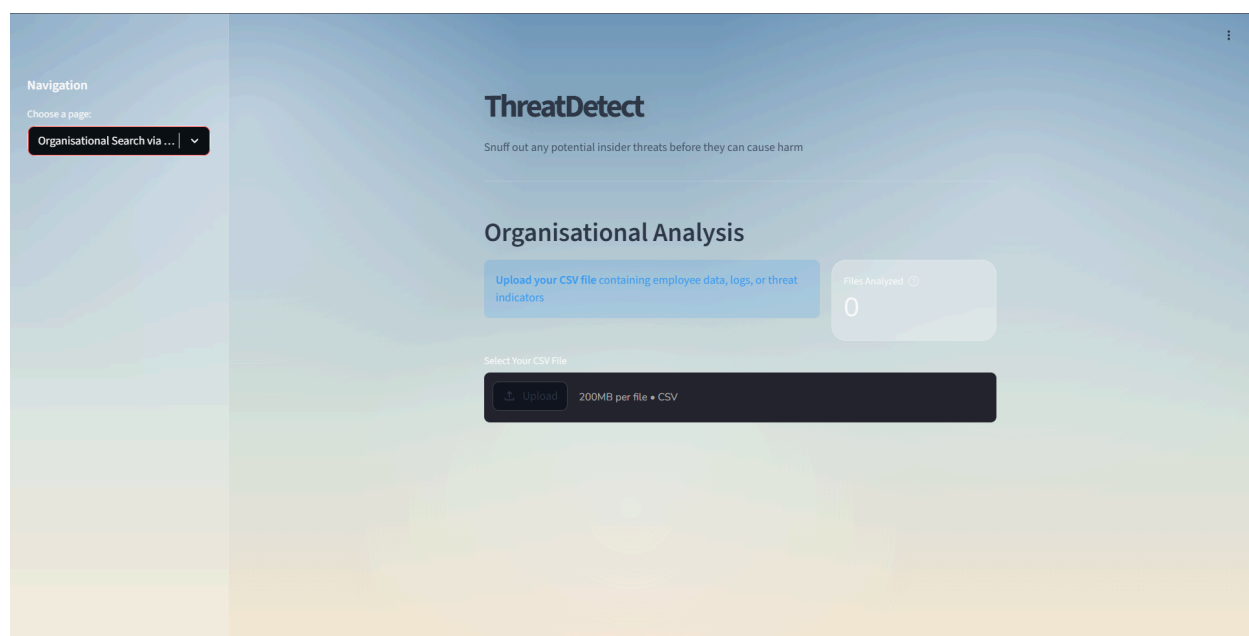
Description of Trained Model Files

`AI\_Model\_Code/insider\_threat\_model.pkl` is the primary trained artifact. It is a Python pickle object containing the full inference pipeline, including:

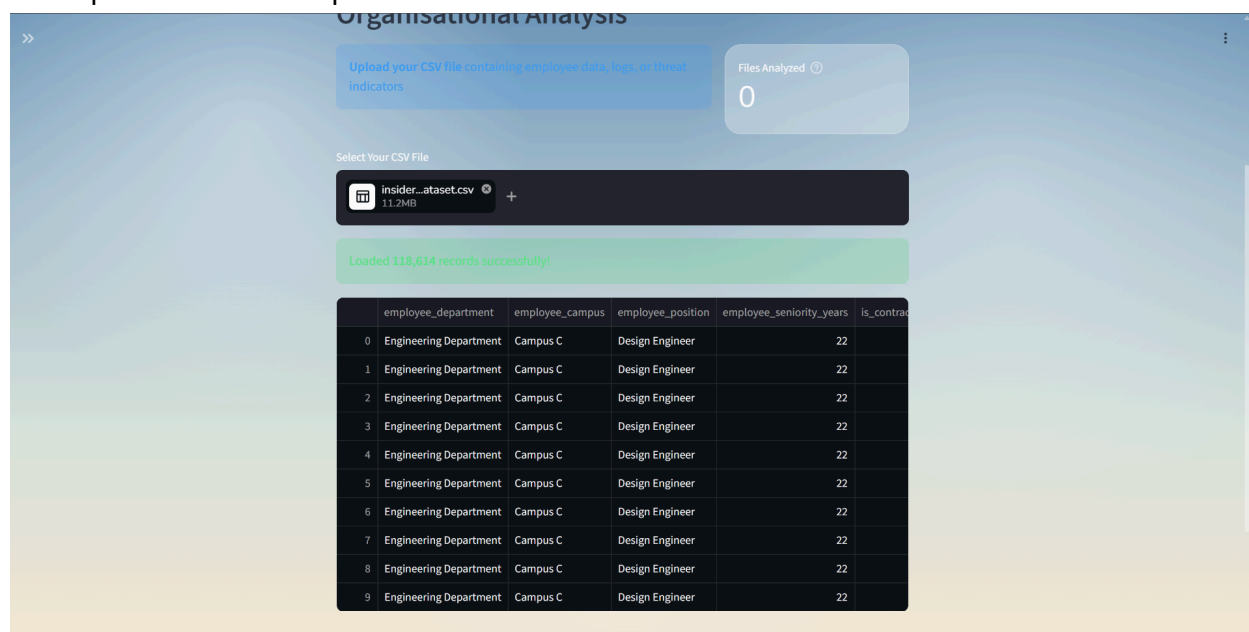
- XGBClassifier for binary insider-threat classification.
- IsolationForest for anomaly score generation.
- Data preprocessing artifacts:
 - label encoders for categorical features
 - standard scaler for numeric inputs
- Model metadata and configuration:
 - feature lists
 - best decision threshold
 - global feature importance values

Screenshots Demonstrating Prototype Functionality

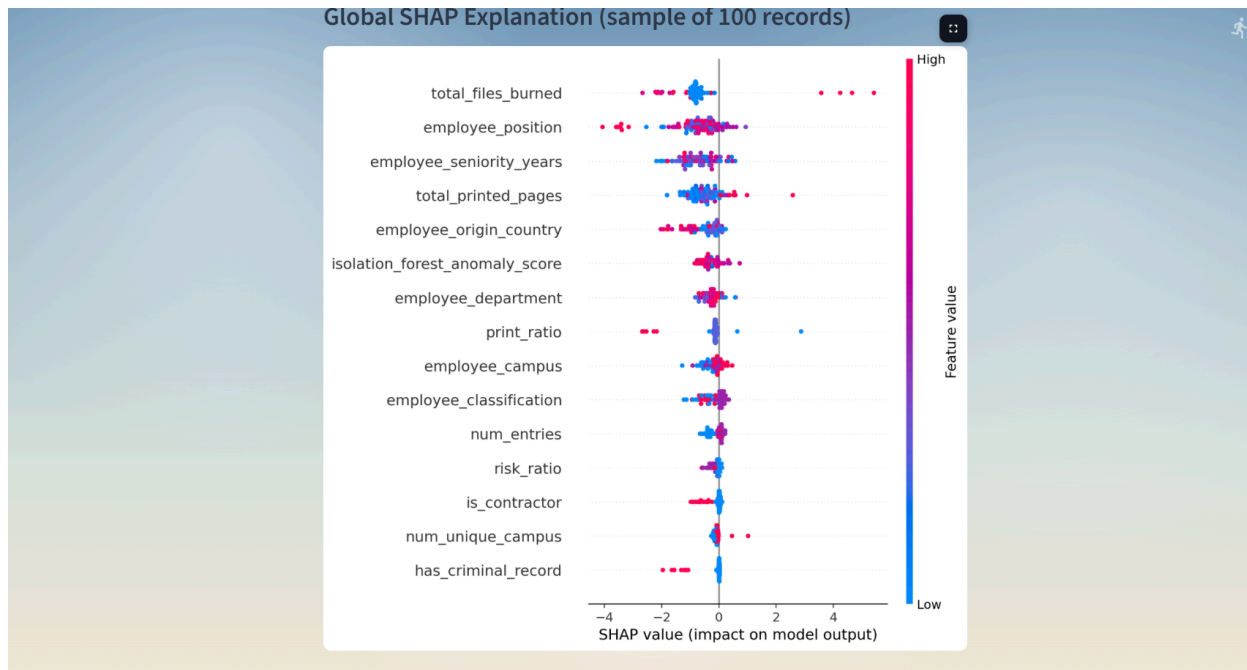
Main Page with navigation bar



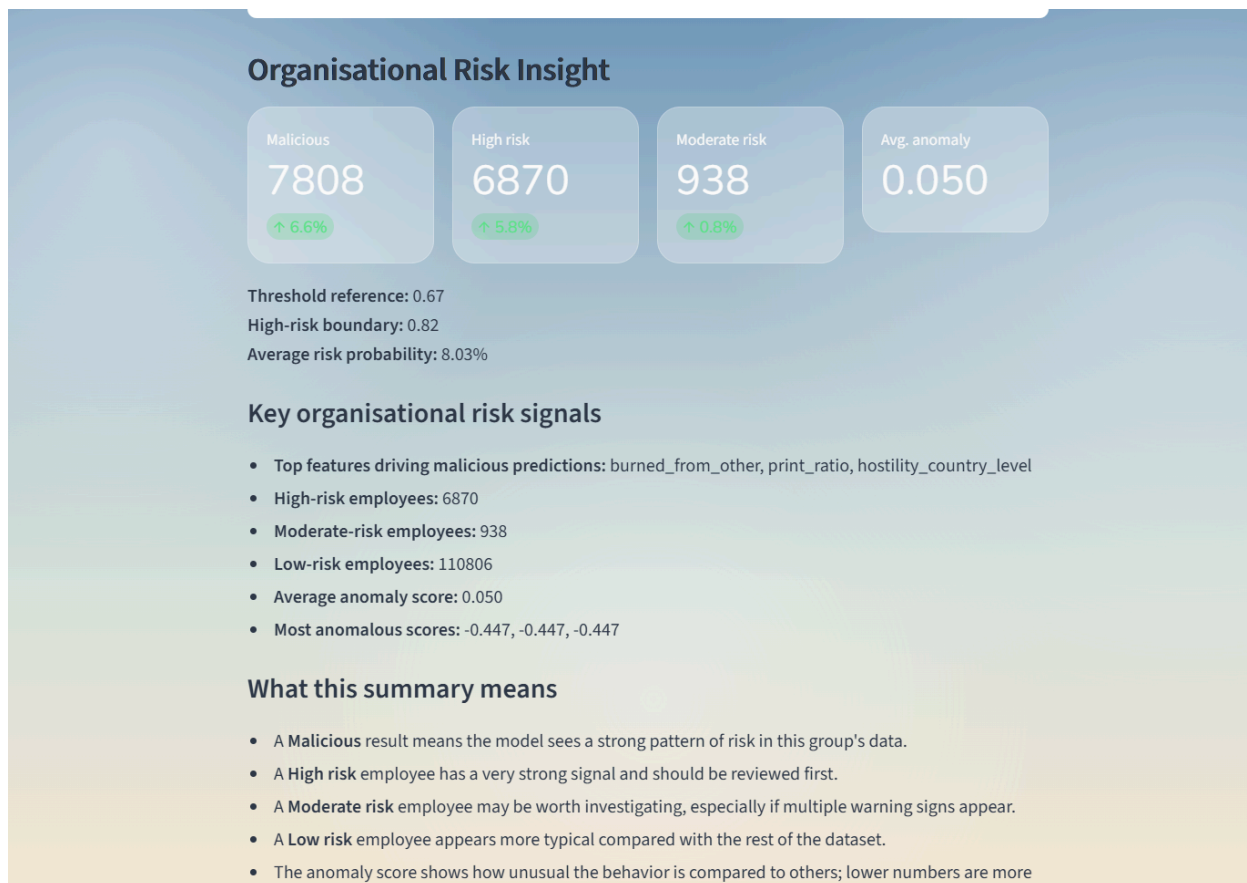
CSV upload and dataset preview



Global Shap Explanation plot



Per Record SHAP Explanation section with human readable points



What this summary means

- A **Malicious** result means the model sees a strong pattern of risk in this group's data.
- A **High risk** employee has a very strong signal and should be reviewed first.
- A **Moderate risk** employee may be worth investigating, especially if multiple warning signs appear.
- A **Low risk** employee appears more typical compared with the rest of the dataset.
- The anomaly score shows how unusual the behavior is compared to others; lower numbers are more unusual.

What to do next

- Start with the high-risk employees and use the per-record explanation section to see the exact factors affecting the decision.
- Pay attention to the top features listed above, such as after-hours activity or unusual access patterns.
- If you are unsure, export the results and share them with your security or compliance team for a deeper review.
- If the dataset is incomplete, add more employee or activity details and rerun the analysis to improve confidence.

7808 employees (6.6%) exhibit malicious patterns. The highest-risk features across the organisation are: burned_from_other, print_ratio, hostility_country_level.

Average confidence for flagged employees is 93.7%.

Navigation

Choose a page:

Exploratory Data Analysis

Quick Summary

	count	unique	top
employee_department	118614	11	R&D Dep
employee_campus	118614	3	Campus
employee_position	118614	48	Secretar
employee_seniority_years	118614	None	None
is_contractor	118614	None	None
employee_classification	118614	None	None
has_foreign_citizenship	118614	None	None
has_criminal_record	118614	None	None
has_medical_history	118614	None	None
employee_origin_country	118614	32	Israel

Missing values: None detected

Detected target column: **is_malicious**

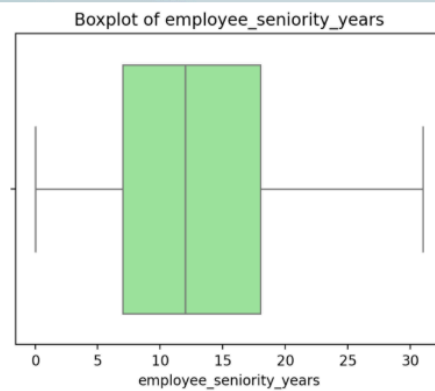
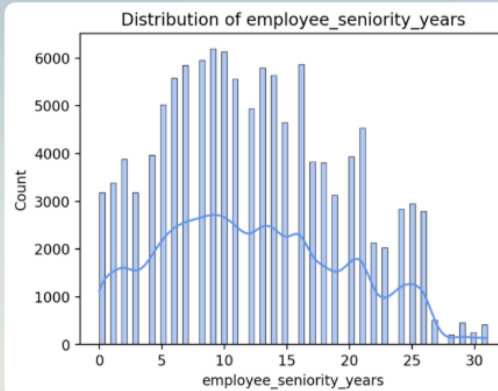
is_malicious	count
0	112228
1	6386

Numeric Feature Exploration

Numeric Feature Exploration

Choose numeric column to inspect

employee_seniority_years



Scatter / Relationship Explorer

X axis

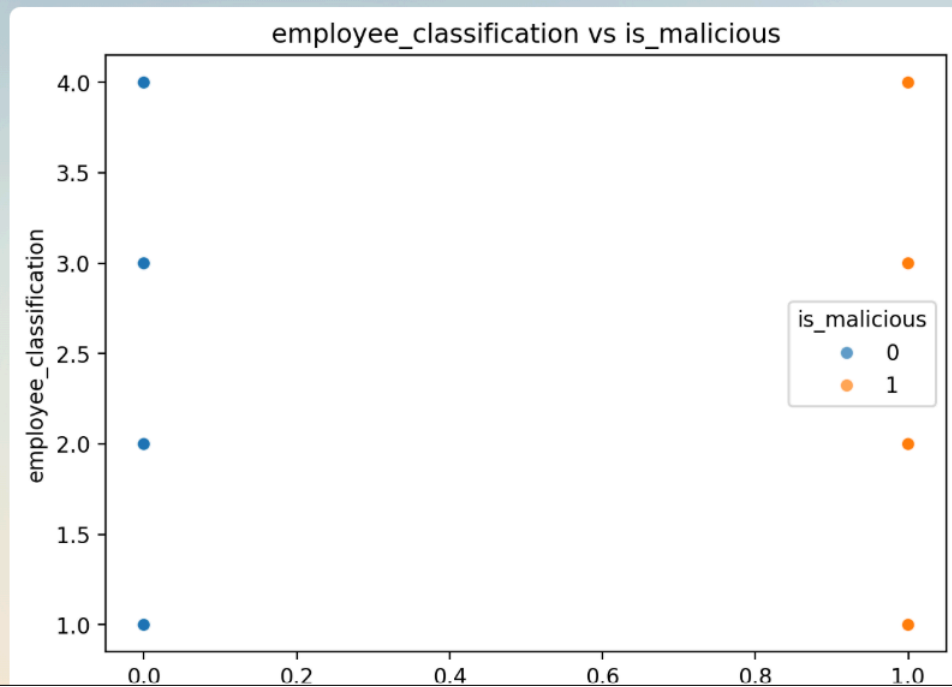
is_malicious

Y axis

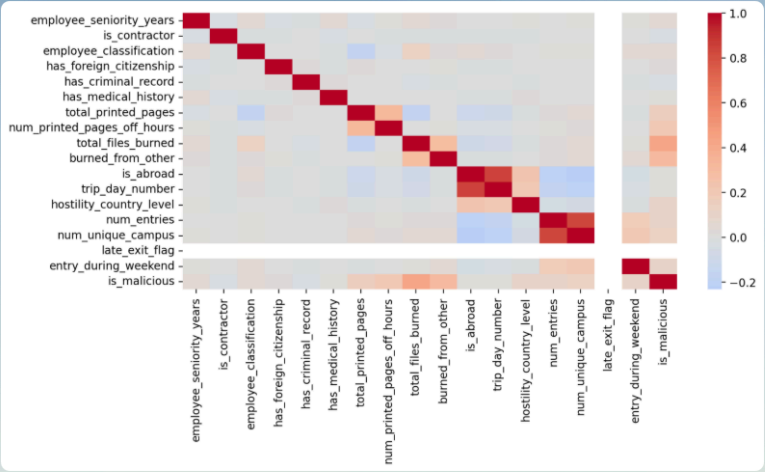
employee_classification

Color by (optional)

is_malicious



Correlation Matrix (numeric)

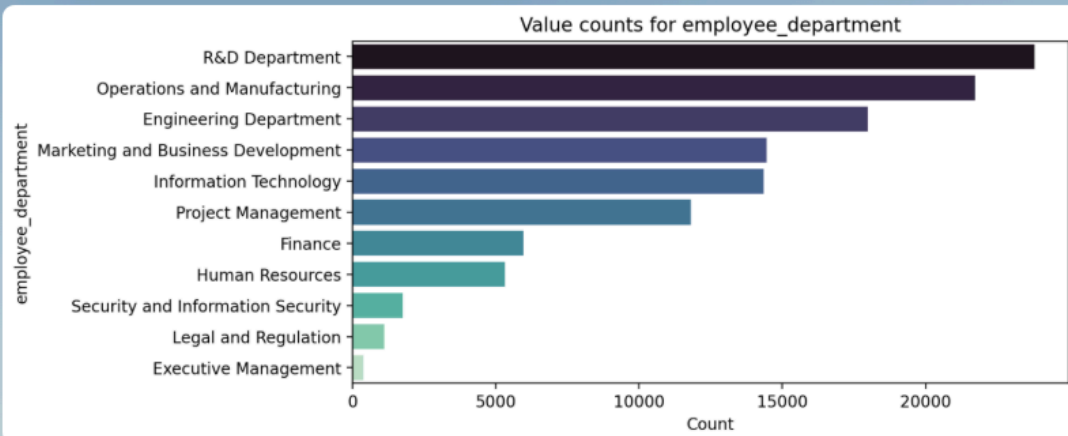


Top features correlated with target

	is_malicious
total_files_burned	0.4235
burned_from_other	0.2975
num_printed_pages_off_hours	0.1969
total_printed_pages	0.1565
num_unique_campus	0.1226
hostility_country_level	0.1095
entry_during_weekend	0.1051

Choose categorical column (counts)

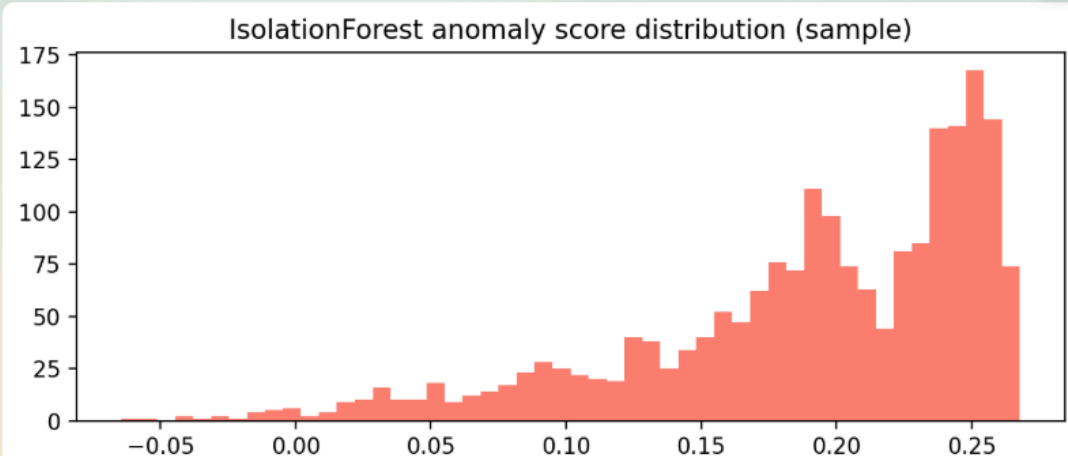
employee_department



Quick Anomaly Scan (IsolationForest)

Run quick anomaly scan

Approx. 20 outliers detected in sampled data (top 1%).



Troubleshooting Notes

Common issues

- ModuleNotFoundError or missing package errors
 - Ensure the Python virtual environment is activated.
 - Install dependencies with `pip install -r requirements.txt`.

- **ValueError: Missing required columns**
 - Uploaded CSV must include the raw feature columns expected by `insider_threat_model.pkl`.
 - Ensure categorical columns are spelled exactly as the model expects.
- **Column contains unseen categories**
 - The uploaded CSV includes a categorical value not present during training.
 - Either clean the data or extend the training pipeline with new categories.
- **SHAP or model load warnings**
 - These can occur when unpickling models saved with an older scikit-learn version.
 - The app may still run, but consider re-training the model and re-saving with the current environment if stability is required.
- **Sample dataset not available**
 - Confirm `AI_Model_Code/insider_threat_clean_dataset.csv` exists in the repository.

Debugging tips

- Use the browser developer console for Streamlit runtime errors.
- Open the terminal where Streamlit is running to view stack traces.
- Confirm the working directory is the repository root when running `streamlit run streamlit_app.py`.